

# Семинар по C++ №8

Sanitizers and GDB

# Вспоминаем проблемы

- Out of range
- Double free
- Memory leak
- Use after free
- Use after return
- ...

# Sanitizers

- There are many of them:
- -fsanitize=address
- -fsanitize=pointer-compare
- -fsanitize=undefined
- ...
- [The full list](#)

# Adress Sanitizer

- How to use?
  - `g++ -fsanitize=address main.cpp`
  - `./a.out`
  - **`ASAN_OPTIONS=detect_stack_use_after_return=1`** `./a.out` – for «use after return» checks
- Why to use?
  - index out of range
  - double free
  - use after free
  - memory leak
  - ...

# Undefined

- How to use?
  - `g++ -fsanitize=undefined main.cpp`
  - `./a.out`
- Why to use?
  - To detect undefined behaviour

# How does it work?

- malloc/free wrappers
- memory access wrappers
- stack
- For more information, read [GitHub](#)

Before:

```
*address = ...; // or: ... = *address;
```

After:

```
if (IsPoisoned(address)) {  
    ReportError(address, kAccessSize, kIsWrite);  
}  
*address = ...; // or: ... = *address;
```

Original code:

```
void foo() {  
    char a[8];  
    ...  
    return;  
}
```

Instrumented code:

```
void foo() {  
    char redzone1[32]; // 32-byte aligned  
    char a[8];         // 32-byte aligned  
    char redzone2[24];  
    char redzone3[32]; // 32-byte aligned  
    int *shadow_base = MemToShadow(redzone1);  
    shadow_base[0] = 0xffffffff; // poison redzone1  
    shadow_base[1] = 0xffffffff00; // poison redzone2, unpoison 'a'  
    shadow_base[2] = 0xffffffff; // poison redzone3  
    ...  
    shadow_base[0] = shadow_base[1] = shadow_base[2] = 0; // unpoison all  
    return;  
}
```

# GDB

- **g++ -g main.cpp** – compile with debug information
- **gdb a.out** – start debugging this executable
- **gdb --args a.out arg1 arg2** – pass command line arguments
  
- **break (b) func\_name/line/...** – set a breakpoint
- **run (r)** – run a program to the end (or the nearest breakpoint)
- **continue (c)** – continue execution to the end (or the nearest breakpoint)
- **quit** – quit the debugger

# Navigation

- **step (s)** – go to the next line (step inside a function)
- **next (n)** – go to the next line (**do not** step inside a function)
- **stepi / nexti** – the same, but to the next assembly instruction
- **finish** – continue until the end of current function
- **return** – return from the function immediately



# GDB (layout)

- **tui enable/disable (Ctrl + X A) / gdb -tui** – запустить tui (Text User Interface)
- **layout next(n)/prev(p)/src/asm/split/regs** - изменить layout
- **focus next(n)/prev(p)/src/asm/split/regs** - изменить фокус
- **tui reg sse/float/...** - добавить окно с определённым layout
  
- **Ctrl + X 2** – layout next
- **Ctrl + P / N** – previous/next command
- **Ctrl + L** – перерисовать экран GDB

# Breakpoints

- **break (b) func\_name/line/...** – set a breakpoint
- **info break** – get the list of breakpoints
- **delete \*break\_num\*** – delete a breakpoint
- **clear** – delete all breakpoints
- **enable / disable \*break\_num\*** – obvious

# Stack

- **backtrace (bt)** –распечатать backtrace
- **backtrace full** – распечатать backtrace и локальные переменные
- **list \*func\_name\*/\*line range\*/...** – распечатать код
- **info args** – распечатать аргументы функции
- **info locals** – распечатать локальные переменные

# utils

- **set var \*var\_name\*=\*value\*** – set a value to a variable
- **gdb --pid \*pid\*** – attach to a working process
- **detach** – detach from a previously attached process
- **set disassembly-flavor intel (att)** (>> ~/.gdbinit)

# GDB (printing memory)

- **set var \*var\_name\*=\*value\*** – установить значение переменной
- **x/nfu addr** - [examine memory](#)
- **p /format \*var\_name\*** – display a variable
- **Format:**
  - a – pointer
  - d – decimal
  - x – hexadecimal
  - f – floating point value
  - ...

# GDB (reverse execution)

- **record** – начать запись исполнения
- **reverse-step / rs** – шаг в обратном направлении (с заходом в функцию)
- **reverse-next / rn** – шаг в обратном направлении (без захода в функцию)
- **reverse-finish** – шаг в обратном направлении до точки вызова функции
- **reverse-continue / rc** – продолжить исполнение до следующего breakpoint

# Core dump

- **`sudo sysctl kernel.core_pattern= "./coredump"`** – установить сохранение корок в текущей директории
- **`ulimit -c unlimited; ./a.out`** – убрать ограничения на размер корок и собрать coredump
- **`gdb ./a.out ./coredump...`** – запустить coredump в отладчике

# Sources

[GDB cheat sheet](#)

[One more](#)

[Занятная статья на Хабре](#)

[TUI documentation](#)

[GDB Essentials Overview](#)